

Arrays, Multidimensional Arrays & Pointers

Q. Define Arrays. Explain one-dimensional arrays, multi-dimensional arrays, and the concept of an array of pointers with syntax and examples.

Definition to Remember

An **Array** is a **contiguous** block of memory.

1. One-Dimensional (1D) Array

* Simplest form - a single list of elements.

* Uses

```
int arr[3] = {10, 20, 30};
```

// Access

2. Multi-Dimensional Arrays

* An array of arrays. Most common: **2D Array** (matrix - rows and columns).

* Uses

```
int matrix[2][2] = {
```

{1, 2},

Memory (Row-Major): 1 | 2 | 3 | 4

Index:

3. Array of Pointers

* An array where each element is a **pointer** (holds a memory address).

* **Syntax**

```
char *names[3] = {"Alice", "Bob", "Charlie"};
```

// name

4. Key Properties of Arrays

| Property | Detail |

| :--- | :-

*****Must-Write Points (for 10 marks)*****

1. Array

*****Quick Recall*****

`Array -
strings)

Algorithmic Implementations: 2D Arrays & Sparse Matrix (C Code)

1. Matrix Multiplication Algorithm & C Code

```
#include <stdio.h>
```

```
#define R1 2
```

```
#define
```

```
void multiplyMatrices(int A[R1][C1], int B[R2][C2], int C[R1][C2]) {
```

```
for (int i
```

```
int main() {
```

```
int A[2][
```

```
multiplyMatrices(A, B, C);
```

```
printf("--- Resultant Product Matrix (2x2) ---\n");
```

```
for (int i
```

2. Sparse Matrix Triplet Representation (C Code)

```
#include <stdio.h>
```

```
struct Element {
```

```
int row;
```

```
struct Sparse {
```

```
int m;
```

```
void createSparse(struct Sparse *s, int matrix[4][4]) {
```

```
s->m =
```

```
for (int i = 0; i < 4; i++) {
```

```
for (int j
```

```
void displaySparse(struct Sparse s) {
```

```
printf("F
```

```
int main() {
```

```
int matr
```

```
struct Sparse s;
```

```
printf("--- Sparse Matrix Triplet Representation ---\n");
```

```
return 0;
```

```
createS
```

```
display
```

```
}
```